GENERALITAT VALENCIANA
Conselleria d'Educació, Cultura i Esport

IES JUAN DE GARAY
C/ Juan de Garay, 25 46017 Valencia
Tel: 961 20 60 45 Fax: 961 20 60 46
46012963@gva.es www.juandegaray.es

GOBIERNO DE ESPAÑA
MINISTERIO DE EDUCACIÓN Y FORMACIÓN PROFESIONAL

Unión Europea
Fondo Social Europeo
El FSE invierte en tu futuro

UD 06.2A. JAVASCRIPT 2

LM

Ciclos Formativos Grado Superior

Pascual Ligero
pa.ligero2@edu.gva.es

2025/2026

Versión:260226.1751

Licencia

Reconocimiento – NoComercial – CompartirIgual (by-nc-sa): No se permite un uso comercial de la obra original ni de las posibles obras derivadas, la distribución de las cuales se debe hacer con una licencia igual a la que regula la obra original.

Nomenclatura

A lo largo de este tema se utilizarán distintos símbolos para distinguir elementos importantes dentro del contenido. Estos símbolos son:

⚠ Importante

⚛ Interesante

ÍNDICE DE CONTENIDO

1. Arrays (Arreglos / Listas) 5
1.1 Métodos Básicos (Mutan la lista) 5
1.2 Métodos Funcionales (Los Reyes del Modern JS) 5
1.3 Novedades ES2023/24 (Inmutabilidad) 6
1.4 Foreach 7
1.5 Errores Típicos 7
1.6 Mini-reto (5 min) 8
1.7 Práctica 8
1.7.1 Objetivo 8
1.7.2 HTML 8
1.8 Javascript 8
2. Objetos 9
2.1 Literales y Acceso Dinámico 9
2.2 Recorrer y Transformar 10
2.3 Agrupamiento (ES2024: Object.groupBy) 10
2.4 Funciones flecha 10
2.4.1 ¿Qué es una Función Anónima? 11
2.4.2 Ejemplos de función anónima 11
2.4.3 Declarar funciones flecha 11
2.4.4 Cuando uso anónimas y cuando flecha 12
2.5 Práctica 12
2.5.1 Objetivo 12
2.5.2 HTML 12
2.5.3 Javascript 13
2.5.4 Javascript de tareas pero con objetos 14
3. Estructuras Avanzadas (Map y Set) 15
3.1 Set (Conjuntos únicos) 15
3.1.1 Crear un conjunto 15
3.1.2 Elimina duplicado de un array 16
3.1.3 Cheet Sheet 16
3.2 Spread 16
3.3 Map (Diccionario Pro) 17
3.3.1 Cheet Sheet 17
3.4 Errores Típicos 17
3.5 Mini-reto (5 min) 18
4. Cierre del Módulo 3 18
4.1 Cheat Sheet 18
4.2 Preguntas Rápidas (V/F) 18
4.3 Checklist de Habilidades 18
5. Ejercicios Finales (La Práctica) 19
5.1 Nivel 1: Calentamiento (Fácil) 19
5.2 Nivel 2: Medio (Intermedio) 19
5.3 Nivel 3: Boss Level (Avanzado) 20

UD06.2A. JAVASCRIPT 2

1. ARRAYS (ARREGLOS / LISTAS)

Un Array es, esencialmente, una lista ordenada. Imagínalo como tu **Cola de Reproducción** en Spotify o la fila para entrar a un concierto. El orden importa y cada elemento tiene una posición numérica (índice) que empieza en 0.

1.1 Métodos Básicos (Mutan la lista)

Estos son los clásicos. Útiles para colas y pilas. **Ojo:** Estos métodos modifican (mutan) el array original.

JavaScript

```
const cola = ["Usuario1", "Usuario2"];

// push: Añade al final (Llega alguien nuevo a la cola)
cola.push("Usuario3");
console.log(cola); // ["Usuario1", "Usuario2", "Usuario3"]

// pop: Saca del final (El último se cansó y se fue)
const ultimo = cola.pop();
console.log(ultimo); // "Usuario3"

// unshift: Añade al principio (El VIP se cuela)
cola.unshift("UsuarioVIP");
console.log(cola); // ["UsuarioVIP", "Usuario1", "Usuario2"]

// shift: Saca del principio (El primero entra al club)
const primero = cola.shift();
console.log(cola); // ["Usuario1", "Usuario2"]
```

1.2 Métodos Funcionales (Los Reyes del Modern JS)

Aquí es donde vivirás el 90% del tiempo. Estos métodos recorren el array por ti. Usaremos esta **Playlist** para los ejemplos:

JavaScript

```
const playlist = [
    { id: 1, titulo: "Bohemian Rhapsody", artista: "Queen", duracion: 354, genero: "Rock" },
    { id: 2, titulo: "Billie Jean", artista: "Michael Jackson", duracion: 294, genero: "Pop" },
```

```
    { id: 3, titulo: "Hotel California", artista: "Eagles", duracion: 390, genero: "Rock" },
    { id: 4, titulo: "Despacito", artista: "Luis Fonsi", duracion: 228, genero: "Pop" }
];
```

- **.map()**: **Transforma**. Entran 4 cosas, salen 4 cosas *transformadas*.
- **.filter()**: **Filtra**. Entran 4 cosas, salen X cosas (las que cumplan la condición).
- **.find()**: **Busca**. Encuentra el *primer* elemento que coincida.
- **.reduce()**: **Reduce**. Entran 4 cosas, sale 1 sola cosa (un total, un resumen).

JavaScript

```
// 1. map: Quiero solo los títulos
const titulos = playlist.map(cancion => cancion.titulo);
console.log(titulos); // ["Bohemian Rhapsody", "Billie Jean", ...]

// 2. filter: Solo canciones de Rock
const rockSongs = playlist.filter(cancion => cancion.genero === "Rock");
console.log(rockSongs); // [{ id: 1, titulo: "Bohemian Rhapsody", artista: "Queen", duracion: 354, genero:
"Rock" },{ id: 3, titulo: "Hotel California", artista: "Eagles", duracion: 390, genero: "Rock" }]

// 3. find: Busca la canción que dura menos de 230 seg
const corta = playlist.find(cancion => cancion.duracion < 230);
console.log(corta); // { id: 4, titulo: "Despacito", artista: "Luis Fonsi", duracion: 228, genero: "Pop"  }

// 4. reduce: ¿Cuánto dura toda la playlist?
// 'total' empieza en 0. 'cancion' es cada item que recorres.
const duracionTotal = playlist.reduce((total, cancion) => {
    return total + cancion.duracion;
}, 0);
console.log(duracionTotal); // 1266

// 5. some & every: Chequeos rápidos (true/false)
const hayReggaeton = playlist.some(c => c.genero === "Reggaeton"); // false (en esta lista)
const sonLargas = playlist.every(c => c.duracion > 200); // true (todas duran > 200)
```

1.3 Novedades ES2023/24 (Inmutabilidad)

En aplicaciones modernas (como React), **no nos gusta cambiar (mutar) el original**. Preferimos crear copias nuevas con los cambios.

- **Mutar (Vieja escuela/Peligroso):** sort, splice.
- **Inmutable (Moderno/Seguro):** toSorted, toSpliced, with.

JavaScript

```
const precios = [100, 50, 200];

// .at(): El método definitivo para buscar índices
console.log(precios.at(-1)); // 200 (El último)

// Ordenar (toSorted vs sort)
let preciosOrdenados = precios.toSorted((a, b) => a - b);
console.log(preciosOrdenados); // [50, 100, 200]
console.log(precios);        // [100, 50, 200] (¡EL ORIGINAL NO CAMBIÓ!)
```

`precios.sort((a, b) => a – b); //muta el originales`
`preciosOrdenados = precios.toSorted((a, b) => a - b);` // Hace una copia y deja como estaba el original!

```
// Reemplazar un valor (with vs asignación directa)
const preciosNuevos = precios.with(1, 999);
// Cambia el índice 1 por 999 en una COPIA
console.log(preciosNuevos); // [100, 999, 200]
```

1.4 Foreach

Para recorrer el array podemos usar los bucles típicos while y for, pero, existe una instrucción que nos hace más fácil ver el contenido sin preocuparnos del tamaño, foreach:

El método `forEach()` recorre cada elemento de un array ejecutando una función callback por cada uno, sin devolver nada (undefined). Es ideal para efectos secundarios como mostrar datos, actualizar DOM o acumular en variables externas, pero no para transformar el array (usa `map` para eso).

```
array.forEach((elemento, indice, array) => {
  // código por cada elemento
});
```

- `elemento: Valor actual (obligatorio).`
- `indice`: Posición (0, 1, 2...) opcional.
- `array`: El array original, opcional.

por ejemplo:

```
const playlist = [
 {id: 1, titulo: 'Bohemian Rhapsody', artista: 'Queen'},
 {id: 2, titulo: 'Billie Jean', artista: 'Michael Jackson'}
];
playlist.forEach(cancion => {
```

```
  console.log(`${cancion.titulo} - ${cancion.artista}`);
});
// Bohemian Rhapsody - Queen
// Billie Jean - Michael Jackson[file:1]
```

1.5 Errores Típicos

- Usar .map cuando quieres filtrar (.filter). Map siempre devuelve un array del mismo tamaño pero no de la misma cantidad de campos (te devolverá undefined o false si intentas filtrar ahí ya que a lo mejor no existe el campo en el nuevo array).
- Olvidar el return en las funciones flecha si abres llaves {}.
    - *Mal:* playlist.map(c => { c.titulo }) (Devuelve undefined).
    - *Bien:* playlist.map(c => c.titulo) (Return implícito).
- Confundir find (devuelve el objeto) con filter (devuelve un array con un objeto dentro).

1.6 Mini-reto (5 min)

Tienes un array de vidas = [3, 1, 0, 5, 2].

1. Usa .filter para quitar los que tienen 0 vidas.
2. A ese resultado, encadénale un .map para multiplicar las vidas restantes por 10.

   Pista: vidas.filter(...).map(...)

1.7 Práctica

1.7.1 Objetivo

Vamos a hacer un gestor de tareas personal, en un array colocaremos las tareas que queremos programar (un TODO de toda la vida)

1.7.2 HTML

De momento va a ser muy básico, e interactuaremos usando el console.log, pero luego lo "pintaremos" y le daremos uso desde el DOM (más adelante, no tengas prisa)

El código completo lo tienes en index.html + tareas01.js

```
<!DOCTYPE html>
```

```
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Document</title>
</head>
<body>
    <h1>Tareas 0.1</h1>
    <div id="Tareas">aquí irán las tares en la versión 0.2</div>
    <script src="js/tareas01.js"></script>
</body>
</html>
```

1.8 Javascript

En la lógica, defino un array para almacenar las notas y para mostrarlas, como sé que va a ser más largo voy haciendo funciones ;)

```
let tareas=["Corregir flex!", "Corregir javascript 1","Corregir javascript 2"]

/*
    pide una tarea y la añade al final de las que tengo pendientes
*/
function agregarTarea(){

    let tarea=prompt("dame una tarea:")
    console.log(tarea)
    tareas.push(tarea)
}

/*
    muestra las tareas pendientes
*/
function mostrarTareas(){
    console.log("Tareas pendientes: ")
    console.log("===================")
    tareas.forEach(element => {
        console.log("[]" +element)
    });
    console.log(tareas)
}

agregarTarea()
mostrarTareas()
```

2. OBJETOS

Si un array es una lista, un objeto es una **ficha descriptiva**. Usan pares clave: valor. Es la forma fundamental de guardar datos complejos (usuarios, productos). Además podemos añadir funciones dentro del objeto que solo influyen en el y me permiten por ejemplo cambiar el estado del mismo (cambiar sus atributos).

2.1 Literales y Acceso Dinámico

La diferencia entre el punto . y los corchetes [] es vital.

JavaScript

```
const streamer = {
    nick: "Ibai",
    plataforma: "Twitch",
    "juego-favorito": "LoL", // Clave con guión requiere comillas
    estadisticas: {
        followers: 15000000,
        subs: 50000
    }
};

// Acceso normal (cuando sabes el nombre de la propiedad)
console.log(streamer.nick); // "Ibai"

// Acceso con corchetes (OBLIGATORIO en 2 casos)
// 1. Claves con espacios o caracteres raros
console.log(streamer["juego-favorito"]);

// 2. Acceso dinámico (La clave está en una variable)
const claveQueBusco = "plataforma";
console.log(streamer[claveQueBusco]); // "Twitch"
// console.log(streamer.claveQueBusco); // ERROR: Busca literalmente "claveQueBusco"
```

2.2 Recorrer y Transformar

A veces necesitas convertir un objeto en array para usar .map o .filter.

JavaScript

```
// Object.keys: Dame las claves
console.log(Object.keys(streamer)); // ["nick", "plataforma", "juego-favorito", "estadisticas"]

// Object.values: Dame los valores
console.log(Object.values(streamer)); // ["Ibai", "Twitch", "LoL", {...}]

// Object.entries: Dame pares [clave, valor]
console.log(Object.entries(streamer));
// [ ["nick", "Ibai"], ["plataforma", "Twitch"] ... ]
```

2.3 Agrupamiento (ES2024: Object.groupBy)

Antes esto era una pesadilla de hacer con reduce. Ahora es magia nativa.

JavaScript

```
const juegos = [
    { titulo: "Elden Ring", genero: "RPG" },
    { titulo: "FIFA", genero: "Deportes" },
    { titulo: "Zelda", genero: "RPG" }
];

// Agrupamos por género
const porGenero = Object.groupBy(juegos, juego => juego.genero);

console.log(porGenero);
/* Resultado:
{
    RPG: [ {titulo: "Elden Ring"...}, {titulo: "Zelda"...} ],
    Deportes: [ {titulo: "FIFA"...} ]
}
*/
```

2.4 Funciones flecha

Para el uso de objetos y funciones dentro de ellas vamos a necesitar introducir un nuevo elemento, la funciones flecha.

Son simplemente **una forma más corta, limpia y moderna de escribir funciones** en JavaScript. Nacieron para que escribamos menos código. Y nos solucionan un problema a la hora de acceder a variables “confusas” desde funciones anónimas.

Lo sé para explicar la función flecha acabo de introducir otra más, la función anónima. Vamos a ello.

2.4.1 ¿Qué es una Función Anónima?

Una función anónima es, literalmente, **una función que no tiene nombre**. Se supone que solo la vas a usar en un momento concreto y puedes olvidarlas.

Normalmente, le ponemos nombre a las funciones (function sumar() { ... }) para poder llamarlas muchas veces en diferentes partes de nuestro código. Pero a veces, solo necesitamos ejecutar un bloque de código *una sola vez* o en un momento muy específico (por ejemplo, cuando alguien hace clic en un botón o cuando pasa un temporizador). Ahí entran las funciones anónimas.

2.4.2 Ejemplos de función anónima

El clásico temporizador (setTimeout)

Le decimos al navegador: "Espera 2 segundos y ejecuta esto". No necesitamos ponerle

nombre a la función porque no la vamos a volver a usar en ningún otro lado.

```
// Usando la palabra 'function' (Sintaxis clásica)
setTimeout(function() {
    console.log("¡Sorpresa tras 2 segundos!");
}, 2000);
```

Una variable

```
// es un poco enrevesado, pero puedes definir una variable para usar una función
anónima

const saludar = function(nombre) {
    return "Hola " + nombre;
};
console.log(saludar("Carlos"))
```

2.4.3 Declarar funciones flecha

Para convertirla en flecha, hacemos dos cosas:

1. Borramos la palabra `function`.
2. Ponemos una flecha => entre los parámetros `(nombre)` y las llaves `{}`.

```
const saludar = (nombre) => {
    return "Hola " + nombre;
};
console.log(saludar("Carlos"))
```

// Otro ejemplo usado antes

const titulos = playlist.map(cancion => cancion.titulo);

o

```
      tareas.forEach(element => {
        element.mostrarEnConsola()
      });
```

```
// Es el ejemplo del temporizador de arriba, pero más moderno
setTimeout(() => {
    console.log("¡Sorpresa tras 2 segundos!");
}, 2000);
```

Por último puede que lo veas así, si la función solo devuelve una instrucción:

```
const saludar = nombre => "Hola " + nombre;
```

2.4.4 Cuando uso anónimas y cuando flecha

- **Las funciones normales (`function() {}`)** si quieres que la función tenga su propio contexto independiente (muy útil a veces en eventos del DOM donde quieres que tener referencia (this) del botón que clickaste).
- **Las funciones flecha (`() => {}`)** cuando quieras usar variables u objetos que están "afuera" de la función sin perder la referencia a ellos, como en métodos de arrays (map, filter, forEach) o temporizadores.

De momento usa funciones flecha, no tienen los problemas de las anónimas y se usa el 90% de las veces. Más adelante las volveremos a utilizar en eventos.

2.5 Práctica

2.5.1 Objetivo

Vamos a mejorar nuestra práctica de tareas pendientes. Cada tarea va a tener mas campos para ser más útil.

Además haremos que cada tarea sea un objeto que tendrá un método que la cambiará entre hecha o no la tarea cada vez que se invoque.

El código completo lo tienes en index02.html + tareas02.js

2.5.2 HTML

Seguimos igual de momento, aún no cambiamos nada

2.5.3 Javascript

// relleno la lista de tareas base con ejemplos

```
let tareas=[
    {id:1, descripcion: "Estudiar arrays: forEach/map", fechaLimite: "2026-02-
23", prioridad: "alta", completada: false},
    {id:2, descripcion: "5 ejercicios con objetos JS", fechaLimite: "2026-02-24",
prioridad: "alta", completada: false},
    {id:3, descripcion: "DOM: Manipular elementos", fechaLimite: "2026-02-25",
prioridad: "media", completada: false},
    {id:4, descripcion: "DOM: Eventos", fechaLimite: "2026-02-26", prioridad:
"baja", completada: false}
    ]

/*
    Devuelve el id para la nueva tarea, busca el máximo y devuelve el siguiente
*/
function devuelveNuevoId(){
    let id=1
```

```
    tareas.forEach(element => {
        if(element.id>id){
            id=element.id
        }
    });
    return id+1
}

/*
    pide una tarea y la añade al final de las que tengo pendientes
*/
function agregarTarea(){
    let descripcion=prompt("dame una tarea:")
    let fechaLimite=prompt("dame una fecha(yyyy-mm-dd):")
    let prioridad=prompt("dame una prioridad (alta | media | baja ):")
    // averigüo la id que me toca
    let id=devuelveNuevoId()
    tareas.push({
        id: id,
        descripcion: descripcion,
        fechaLimite: fechaLimite,
        prioridad: prioridad,
        completada: false
    });
}

/*
    muestra las tareas pendientes
*/
function mostrarTareas(){
    console.log("Tareas pendientes: ")
    console.log("===================")
    tareas.forEach(element => {
        console.log("[]" +element)
    });
    console.log(tareas)
}

agregarTarea()
mostrarTareas()
```

Perfecto, pero esto no se diferencia de las listas de antes, ¿no?.

2.5.4 Javascript de tareas pero con objetos

El código completo lo tienes en index02b.html + tareas02B.js

Vamos a hacer un array de **objetos** tarea, estas tareas serán igual que antes, pero además podrán tener unas funciones que harán algo con sus valores.

En nuestro caso cada tarea tendrá los valores (id, descripcion, fechaLimite, prioridad) y dos funciones (toggleCompletada() , mostrarEnConsola())

- toggleCompletada(): Cambia el estado de la tarea entre pendiente y completada.

- MostrarEnConsola(): Muestra en consola los campos de la tarea.

El **objeto** sería de este estilo:

```
//Definimos un objeto tareas que tendrá atributos y funciones.
let tareaEjemplo = {
  id: 2,
  descripcion: "5 ejercicios con objetos JS",
  completada: false,
  fechaLimite: "2026-02-25",
  prioridad: "alta",
  toggleCompletada: () => {
      tareaEjemplo.completada = !tareaEjemplo.completada;
      return this;
    },
  mostrarEnConsola: () => {
      console.log(`- (${tareaEjemplo.id}) ${tareaEjemplo.completada ? '[X]' :
'[]'} :  ${tareaEjemplo.descripcion} : ${tareaEjemplo.fechaLimite}  ($
{tareaEjemplo.prioridad}) `);
    }
};
```

y la definición de la **lista** de tareas seria:

```
//creo las tareas de ejemplo
tareas=[ tareaEjemplo, tareaEjemplo2, …..]
```

Esto funcionaría, pero tenemos que copiar el let tareaEjemplo con distintos nombres para cada tarea que queramos añadir …. ¡¡¡un montón de código repetido!!!!. Podemos mejorarlo generando una función que cree los objetos recibiendo los parámetros y genere el objeto:

```
function crearTarea(id, descripcion, fechaLimite, prioridad) {
  const tarea = {
    id,
    descripcion,
    completada: false,
    fechaLimite,
    prioridad
  };

  // usando funciones anonimas flecha es mas sencillo
  // cambiar entre completada o no
  tarea.toggleCompletada = () => {
    tarea.completada = !tarea.completada;
    return tarea;
  };

  // Mostrar en consola
  tarea.mostrarEnConsola = () => {
    const estado = tarea.completada ? '[X]' : '[ ]';
    console.log(`- (${tarea.id}) ${estado} :  ${tarea.descripcion} : $
{tarea.fechaLimite}  (${tarea.prioridad}) `);
  };
```

```
  return tarea;
}


//creo las tareas
tareas=[
    crearTarea(1, "Estudiar arrays: forEach/map", "2026-02-23","alta"),
    crearTarea(2, "5 ejercicios con objetos JS", "2026-02-24", "alta"),
    crearTarea(3, "DOM: Manipular elementos", "2026-02-25", "media"),
    crearTarea(4, "DOM: Eventos", "2026-02-26", "baja")
    ]
```

3. ESTRUCTURAS AVANZADAS (MAP Y SET)

A veces Arrays y Objetos no son suficientes o no son eficientes.

3.1 Set (Conjuntos únicos)

Un conjunto es una lista donde **NO** se permiten duplicados. Ideal para limpiar datos.

3.1.1 Crear un conjunto

```
// crear
const amigosDiscord = new Set();

// añadir elementos
amigosDiscord.add("Alex");
amigosDiscord.add("Sara");
amigosDiscord.add("Alex");  // Ignorado
amigosDiscord.add("Pablo");

console.log(`Tienes ${amigosDiscord.size} amigos únicos`);
// Tienes 3 amigos únicos

// Quitar amigo tóxico
amigosDiscord.delete("Alex");
```

3.1.2 Elimina duplicado de un array

```
const etiquetas = ["javascript", "css", "javascript", "html", "css"];

// Crear un Set elimina los repetidos automáticamente
const unicosSet = new Set(etiquetas);

// Convertirlo de vuelta a Array (Spread)
const unicosArray = [...unicosSet];

console.log(unicosArray); // ["javascript", "css", "html"]
```

3.1.3 Cheet Sheet

Acción	Código	Resultado
Insertar	`set.add("elemento")`	Añade SI NO existe
Eliminar	`set.delete("elemento")`	Quita para siempre
Tamaño	`set.size`	Cuántos únicos
Existe	`set.has("elemento")`	true/false

3.2 Spread

El **operador spread** (...) **"desempaqueta"** arrays y objetos, como **abrir una caja** y sacar todo su contenido.

```
const likes = [100, 200, 150];
const nuevoPost = [...likes, 500];  // Añade nuevo post SIN tocar originales
console.log(nuevoPost); // [100, 200, 150, 500]
```

3.3 Map (Diccionario Pro)

Parecido a un Objeto, pero las claves pueden ser cualquier cosa (números, booleanos, objetos) y mantiene el orden de inserción. Es más rápido para añadir/quitar cosas frecuentemente.

```
cconst inventario = new Map();

// clave = nombre o id del producto
inventario.set("tornillos", { precio: 0.12, cantidad: 100 });
inventario.set("tuercas",   { precio: 0.08, cantidad: 200 });

// método “pegado” al Map (propiedad normal del objeto)
inventario.totalValor = function () {
  let total = 0;
  for (const item of this.values()) total += item.precio * item.cantidad;
  return total;
};

console.log(inventario.get("tornillos")); // { precio: 0.12, cantidad: 100 }
```

```
console.log(inventario.totalValor());     // 28
```

3.3.1 Cheet Sheet

```
const mapa = new Map();           // Vacío
const mapa = new Map([[1,'uno'], [2,'dos']]);  // Con datos
```

Acción	Código	Resultado
Añadir	`mapa.set('clave', 'valor')`	Añade/sobreescribe
Obtener	`mapa.get('clave')`	Valor o `undefined`
Tamaño	`mapa.size`	Número de pares
Eliminar	`mapa.delete('clave')`	`true` si existía
Existe	`mapa.has('clave')`	`true/false`
Vaciar	`mapa.clear()`	Mapa vacío

3.4 Errores Típicos

- Intentar acceder a un Map con corchetes: mapa[101] // (Devolverá undefined). Debes usar .get(101) .
- Pensar que Set tiene índices. miSet[0] no funciona.

3.5 Mini-reto (5 min)

Crea un array con números repetidos [1, 5, 1, 5, 10]. Usa new Set() y el spread operator para obtener [1, 5, 10] en una sola línea.

4. CIERRE DEL MÓDULO 3

4.1 Cheat Sheet

- **Añadir/Quitar (Array):** push (final), pop (final), shift (inicio).
- **Transformar:** .map(item => nuevoItem).
- **Filtrar:** .filter(item => condicion).

- **Acumular:** .reduce((acc, item) => ..., inicio).
- **No Mutar:** Usa toSorted() en vez de sort().
- **Agrupar:** Object.groupBy(array, criterio).
- **Copiar/Fusionar:** Spread operator ....
- **Únicos:** new Set(array).

4.2 Preguntas Rápidas (V/F)

1. .map() modifica el array original. (V/F)
2. push() añade un elemento al principio del array. (V/F)
3. Object.keys() devuelve un array con los valores del objeto. (V/F)
4. El Spread Operator ... sirve para copiar objetos. (V/F)
5. const impide que modifiques el contenido de un array (ej. con push). (V/F)
6. Set permite guardar valores duplicados. (V/F)
7. Para buscar un elemento por ID usamos .find(). (V/F)
8. Object.groupBy devuelve un objeto con arrays dentro. (V/F)
9. Si uso let [a] = [10, 20], a vale 10. (V/F)
10. toSorted() es la versión inmutable de sort(). (V/F)

4.3 Checklist de Habilidades

- [ ] Sé recorrer un array y transformarlo con .map.
- [ ] Sé filtrar datos basura con .filter.
- [ ] Entiendo la diferencia entre cliente.nombre y cliente["nombre"].
- [ ] Sé usar Destructuring para sacar datos limpiamente.
- [ ] Sé eliminar duplicados de una lista con Set.

5. EJERCICIOS FINALES (LA PRÁCTICA)

Dataset para los ejercicios (Cópialo en tu consola):

JavaScript

```
const carrito = [
    { id: 1, nombre: "Teclado Mecánico", precio: 120, cantidad: 1, categoria: "Periféricos" },
    { id: 2, nombre: "Monitor 24", precio: 200, cantidad: 2, categoria: "Pantallas" },
    { id: 3, nombre: "Ratón Gamer", precio: 50, cantidad: 1, categoria: "Periféricos" },
    { id: 4, nombre: "Cable HDMI", precio: 10, cantidad: 3, categoria: "Cables" }
];

const usuarios = [
    { id: 1, nick: "Slayer", skills: ["JS", "React"] },
    { id: 2, nick: "Coder123", skills: ["Python", "JS"] },
    { id: 3, nick: "NoobMaster", skills: [] }
];
```

5.1 Nivel 1: Calentamiento (Fácil)

1. **Nombres:** Usa .map para obtener solo un array con los nombres de los productos del carrito.
    - *Output:* ["Teclado Mecánico", "Monitor 24"...]
2. **Baratos:** Usa .filter para obtener los productos que cuesten menos de 100.
3. **Buscador:** Usa .find para encontrar el producto con id: 2.
4. **Stock:** Usa .some para ver si hay algún producto con cantidad > 2. (True/False).
5. **Desempaquetar:** Usa destructuring para sacar el nick del primer usuario del array usuarios y guardarlo en una variable llamada nombreAdmin.
    - *Pista:* const [{...}] = ...
6. **Spread:** Crea un nuevo array carritoActualizado que tenga todos los productos del carrito + uno nuevo { id: 5, nombre: "Alfombrilla", ... } usando Spread.

5.2 Nivel 2: Medio (Intermedio)

7. **Total a Pagar:** Usa .reduce para calcular el precio total del carrito (teniendo en cuenta la cantidad de cada uno).
    - *Pista:* acumulador + (item.precio * item.cantidad)
8. **Inventario por Categoría:** Usa Object.groupBy para agrupar el carrito por categoria.
9. **Fusión de Datos:** Tienes un objeto infoAdicional = { region: "EU", premium: true }. Crea un nuevo objeto para el usuario 1 que combine sus datos originales + infoAdicional usando Spread.

10. **Última Skill:** Usa .flatMap (o map + flat) para hacer una lista de TODAS las skills de todos los usuarios, y luego .at(-1) para ver la última.
11. **Actualización Segura:** Usa .map sobre el carrito. Si el producto es "Cable HDMI", devuelve un nuevo objeto con el precio aumentado a 15 (sin tocar el resto de propiedades). Si no, devuelve el objeto tal cual.
12. **Ordenar:** Usa .toSorted para ordenar el carrito por precio de mayor a menor.

5.3 Nivel 3: Boss Level (Avanzado)

13. El Reporte de Skills Únicas:

- **Enunciado:** Quiero un array con todas las skills que existen en usuarios, pero **sin repetir** ninguna.
- **Restricción:** Usa .flatMap (o map y flat), luego Set y luego Spread.
- **Output:** ["JS", "React", "Python"]
- *Pista:* Recuerda que Set limpia duplicados.

14. Diccionario de Productos (Map):

- **Enunciado:** Convierte el array carrito en un Map donde la **clave** sea el id y el **valor** sea el objeto del producto.
- **Restricción:** Usa un bucle for...of o .forEach para llenar el Map.
- **Output:** Map(4) { 1 => {...}, 2 => {...} ... }
- *Pista:* miMap.set(producto.id, producto)

15. Análisis de Presupuesto:

- **Enunciado:** Encadena métodos.
  1. Filtra productos de categoría "Periféricos".
  2. Obtén sus precios totales (precio * cantidad).
  3. Súmalos todos.
- **Output:** (120 * 1) + (50 * 1) = 170.
- *Pista:* filter().reduce() o filter().map().reduce().

16. Clonación Profunda (Reto Mental):

- **Enunciado:** Si usas const copia = [...carrito], has hecho una copia

superficial. Si cambias copia[0].precio = 9999, ¿cambia también en el carrito original? Pruébalo.

- **Reto:** ¿Cómo harías una copia real donde los objetos de dentro también sean nuevos? (Puedes usar structuredClone(carrito) si tu navegador es moderno, o JSON.parse(JSON.stringify(carrito))).